LandCover.ai Semantic Segmentation Pipeline

A file-by-file walkthrough of the U-Net training & inference codebase

---

This document walks through every script in **src_landcover_ai/**, in the order they run, explaining what each file is responsible for, what each function/class does, and how data flows between them. It covers 13 files: config, sanity checks, preprocessing, augmentation, the dataset loader, the model, the loss function, metrics, training, prediction, and the two visualization scripts.

Dataset	LandCover.ai — 5 classes: background, building, woodland, water, road
Task	Semantic segmentation — classify every pixel of a satellite tile
Model	SimpleUNet — a small U-Net (3 down/up steps) built from scratch in PyTorch
Tile size	256 x 256, resized from the original 512 x 512 split output
Framework	PyTorch, with optional CUDA + mixed precision (AMP)

How the pieces fit together

Two kinds of files here: **support modules** (config, preprocessing, augmentation, dataset, model, losses, metrics) that get *imported* by other scripts and never run directly, and **entry-point scripts** that you actually run from the terminal, in this order:

1	**check_mask_values.py** Confirms mask pixels really are class ids 0-4, not raw 0-255 grayscale. Run this before anything else.
2	**compute_class_weights.py** Samples 500 training tiles, counts how many pixels belong to each class, and prints weights to correct the class imbalance. Paste the output into config.py.
3	**visualize.py** Quick sanity check — plots a few image/mask pairs so you can eyeball that loading and label colors look right before spending time training.
4	**train.py** The actual training loop. Trains SimpleUNet for NUM_EPOCHS, evaluates on the validation set each epoch, and saves the best checkpoint.
5	**predict.py** Loads the saved checkpoint and either runs inference on the whole test split (with metrics) or on one arbitrary image.
6	**visualize_predictions.py** Picks random test tiles and shows image / prediction / ground-truth side by side, to visually judge how good the model is.

The support modules (config.py, preprocessing.py, augmentation.py, dataset.py, model.py, losses.py, metrics.py) don't appear in that run order because nobody runs them directly — they're imported into steps 2-6 wherever needed. Each still gets its own section below, in roughly the order the data passes through them.

config.py

The single source of truth for every path, hyperparameter, and constant — nothing else hardcodes these.

Every other file in the project does `from config import ...` instead of hardcoding paths or numbers. That means changing one value here (say, BATCH_SIZE or IMAGE_SIZE) changes it everywhere at once, and there's one place to look when something needs tuning.

Path setup

`SRC_DIR` and `PROJECT_ROOT` are computed from `__file__` rather than typed out, so the project still works no matter which machine or folder it's cloned into. From there, every other path — the dataset folder, the train/val/test list files, checkpoints, outputs, visualizations, logs — is built by joining onto `PROJECT_ROOT`. `TILES_DIR` specifically points at the flat folder that `split.py` (the LandCover.ai preprocessing script, not part of this pipeline) fills with individual 512x512 image/mask tiles.

Image & class settings

`IMAGE_SIZE = 256` — every tile gets resized down to 256x256 before it reaches the model (see preprocessing.py). `NUM_CLASSES = 5` and `CLASS_COLORS` together define the label scheme: background, building, woodland, water, road, each mapped to an RGB color used only for human-readable visualization (the actual mask files store plain integers 0-4, not these colors).

CLASS_WEIGHTS

A hardcoded list of 5 floats, one per class, fed into the loss function. LandCover.ai is very imbalanced — woodland covers vastly more pixels than building — so without these weights the model could get a deceptively low loss just by predicting 'woodland' everywhere. This list is meant to be generated by compute_class_weights.py and pasted in here; if left as `None`, training falls back to plain unweighted CrossEntropyLoss.

Training settings

`BATCH_SIZE` (16), `NUM_EPOCHS` (30), and `LEARNING_RATE` (1e-3) control the training loop directly. `NUM_WORKERS = 4` spins up parallel CPU processes to load/resize the next batch of images while the GPU is busy on the current one, so the GPU doesn't sit idle waiting on disk I/O. `USE_AMP = True` turns on automatic mixed precision — using 16-bit floats for parts of the forward/backward pass — which speeds up training and reduces memory use on CUDA GPUs, and is automatically ignored on CPU.

DEVICE

`DEVICE = "cuda" if torch.cuda.is_available() else "cpu"` — every tensor and the model itself get moved to whichever device this resolves to, so the same code runs on a laptop CPU or a CUDA GPU without edits.

check_mask_values.py

A 20-line sanity check you run first, before touching train.py.

The entire pipeline assumes each mask PNG stores the class id *directly as the pixel value* — a pixel with value 2 means 'this pixel is woodland', not an RGB color that needs decoding. This script exists purely to confirm that assumption before you invest time in training.

What it does

It reads the first 5 names from `train.txt`, opens each corresponding `<name>_m.png` mask file in grayscale mode (`convert("L")`), and prints the sorted set of unique pixel values found in it via `np.unique()`. If everything is set up correctly, you should only ever see values from `[0, 1, 2, 3, 4]`. If instead you see something like 0-255 or only 0/255, the masks are encoded differently than this codebase expects, and dataset.py / preprocessing.py would need to be rewritten to decode them (e.g. an RGB-to-id color lookup) before training would produce anything meaningful.

compute_class_weights.py

Measures class imbalance and generates the CLASS_WEIGHTS list for config.py.

Run once, before train.py, to answer: 'how skewed are the classes, and how much should the loss function be corrected for it?' It only samples 500 of the ~7500 available training tiles — enough for a statistically stable estimate, while staying fast.

What it does, step by step

- Reads all tile names from train.txt, then takes a random sample of up to 500 of them (seeded with 42 for reproducibility).
- For each sampled tile's mask, counts how many pixels belong to each of the 5 classes using `np.sum(mask == class_id)`, accumulating totals in `pixel_counts`.
- Prints a small table of each class's name, raw pixel count, and share of all sampled pixels — this is the human-readable imbalance report (e.g. woodland dominating vs. building being rare).
- Computes `frequencies = pixel_counts / total_pixels`, then inverse-frequency weights as `1.0 / (frequencies + 1e-6)` — rare classes get a large weight, common classes get a small one. The `1e-6` just avoids a divide-by-zero if a class never appeared in the sample.
- Normalizes those weights so they average to 1.0 (`raw_weights / raw_weights.mean()`) — this keeps the overall loss magnitude comparable to an unweighted loss, so LEARNING_RATE doesn't need retuning just because weights were added.
- Prints the final list formatted so it can be copy-pasted straight into config.py as CLASS_WEIGHTS.

preprocessing.py

Four small, pure functions that turn a raw PIL image/mask into model-ready arrays.

These functions are the shared building blocks used by both dataset.py (loading training data) and predict.py (loading a one-off image for inference), so resizing/normalization logic only exists in one place.

`resize_image(image: Image.Image) -> Image.Image`

- Resizes a photo to IMAGE_SIZE x IMAGE_SIZE using bilinear interpolation.
- Bilinear is fine for photos — colors are continuous, so blending two neighboring pixel colors during resize still produces a sensible in-between color.

`resize_mask(mask: Image.Image) -> Image.Image`

- Resizes a mask to the same IMAGE_SIZE, but with NEAREST-neighbor interpolation instead.
- This is a deliberate and important difference from resize_image: a mask stores class labels (0-4), and 'blending' two labels (e.g. averaging building=1 and water=3 into 2) would silently create a nonsense class. Nearest-neighbor just copies the closest original pixel's label, keeping every value a valid class id.

`normalize_image(image_array: np.ndarray) -> np.ndarray`

- Casts the array to float32 and divides by 255.0, mapping raw 0-255 pixel values into the 0-1 range neural networks train more stably on.

`clamp_mask_ids(mask_array: np.ndarray) -> np.ndarray`

- Casts the mask to int64 (the dtype PyTorch's CrossEntropyLoss expects for target labels).
- Then zeroes out any pixel whose value falls outside the valid [0, NUM_CLASSES) range — a safety net in case resizing artifacts or a stray corrupted pixel produced an invalid class id, which would otherwise crash the loss function.

augmentation.py

One function: randomly flip an image and its mask together, to artificially grow the training set.

`random_flip(image: np.ndarray, mask: np.ndarray)`

- With 50% probability, flips both the image and mask left-right (np.fliplr).
- Independently, with another 50% probability, flips both up-down (np.flipud).
- The key detail: image and mask are flipped by the exact same random decisions, in the same calls — if the image is mirrored, the mask must be mirrored identically, or the labels would no longer line up with the pixels they describe.
- Satellite tiles have no fixed 'up' direction the way photos of people or objects do, so flipping doesn't create anything physically unrealistic — it's a cheap, safe way to show the model more orientations of the same terrain without collecting more data.
- .copy() is called after each flip because np.fliplr/flipud return a view with negative strides, which PyTorch's from_numpy can't safely wrap into a tensor.

dataset.py

Defines SegmentationDataset — bridges the raw tile files on disk and PyTorch's DataLoader.

This is where preprocessing.py and augmentation.py actually get used. It implements PyTorch's `Dataset` interface, which just means providing `__len__` and `__getitem__` — PyTorch's DataLoader (used in train.py and predict.py) handles batching, shuffling, and parallel loading on top of whatever this class returns for a single index.

SPLIT_FILES dict

Maps the friendly split names `"train" / "valid" / "test"` onto the actual list files config.py points to. Note the project intentionally calls it `"valid"` even though the underlying file is `val.txt` (that's just what LandCover.ai's own split.py script names it) — this keeps naming consistent with the rest of the codebase.

`__init__(self, split, augment=False)`

- Looks up the right list file for the given split and reads it into self.names — a plain list of tile name strings (no extension), one per line.
- Stores split and augment (whether random_flip should be applied) for later use.

`__len__(self)`

- Returns the number of tiles in this split, i.e. len(self.names). DataLoader uses this to know how many samples exist and how to build batches/epochs.

`__getitem__(self, index)`

- Looks up the tile name at this index, then loads <name>.jpg as the image and <name>_m.png as the mask from TILES_DIR.
- Runs the image through resize_image() then normalize_image() (from preprocessing.py).
- Runs the mask through resize_mask() then clamp_mask_ids().
- If self.augment is True, applies random_flip() to the image/mask pair together.
- Converts both NumPy arrays into PyTorch tensors: the image is permuted from (H, W, 3) to (3, H, W) — PyTorch convolutions expect channels first — and cast to float; the mask is cast to long (int64), the dtype CrossEntropyLoss requires for targets.
- Returns the (image_tensor, mask_tensor) pair, which is exactly what train.py's and predict.py's DataLoader loops unpack as `images, masks`.

One more detail worth noting: unlike a companion DeepGlobe pipeline this project mirrors, LandCover.ai ships real ground-truth masks for every split including test, so there's no missing-mask edge case to special-case here — every split behaves identically in this class.

model.py

Defines SimpleUNet — the neural network architecture itself, built from scratch.

`conv_block(in_channels, out_channels)`

- A small helper that returns nn.Sequential of: Conv2d -> ReLU -> Conv2d -> ReLU, both convolutions using a 3x3 kernel with padding=1 (so spatial size doesn't shrink from the convolution itself).
- This 'two convs + ReLU' pattern is the repeated building block used at every stage of the U-Net, so it's factored out once instead of duplicated 7 times.

class SimpleUNet(nn.Module)

A compact U-Net: an **encoder** that repeatedly shrinks the image while learning increasingly abstract features (*what* is in the image), a **bottleneck** at the most compressed point, and a **decoder** that grows the resolution back up while relearning precise pixel locations (*where* things are) — helped by **skip connections** that pass fine detail straight from the encoder to the matching decoder stage.

`__init__(self, num_classes)`

- Encoder: enc1 (3→16 channels), enc2 (16→32), enc3 (32→64) — each a conv_block. self.pool is a 2x2 max-pool applied between them, halving height/width each time.
- Bottleneck: one more conv_block, 64→128 channels, at 1/8 the original resolution — the most compressed representation.
- Decoder: three (ConvTranspose2d, conv_block) pairs that mirror the encoder in reverse — up3/dec3 (128→64), up2/dec2 (64→32), up1/dec1 (32→16). Each ConvTranspose2d upsamples resolution by 2x; each following conv_block takes double the channels because the upsampled features get concatenated with the matching encoder output (the skip connection) before convolving.
- final: a 1x1 Conv2d that turns the last 16 feature channels into num_classes channels — one raw score per class, per pixel. This is passed straight into config's NUM_CLASSES (5) at construction time in train.py/predict.py.

`forward(self, x)`

- Encoder pass: e1 = enc1(x) at full resolution; e2 = enc2(pool(e1)) at 1/2 res; e3 = enc3(pool(e2)) at 1/4 res.
- Bottleneck: b = bottleneck(pool(e3)) at 1/8 res.
- Decoder pass, each step upsamples, concatenates with the same-resolution encoder output along the channel dimension (torch.cat), then runs the conv_block — this is what lets the network recover sharp object boundaries instead of a blurry blob, since the encoder features it's borrowing were computed before any downsampling lost detail.
- Returns self.final(d1), a raw-logit tensor of shape (batch, num_classes, H, W) — no softmax is applied here, because the loss function applies it internally (see losses.py).

Note in the code's own comment: this is the same architecture reused from a companion DeepGlobe project, with only num_classes differing — 5 here instead of DeepGlobe's 7.

losses.py

One factory function that builds the (optionally class-weighted) loss.

`get_loss_function(class_weights=None, device="cpu")`

- Returns an nn.CrossEntropyLoss instance — the standard choice for multi-class, per-pixel classification. It compares the model's raw per-pixel logits against the integer class-id mask, and it applies softmax internally as part of its math.
- Important consequence: the model must NEVER apply softmax before this loss — model.py's forward() correctly returns raw logits, not probabilities, for exactly this reason.
- If class_weights is provided (the CLASS_WEIGHTS list from config.py), it's converted into a float32 tensor and moved to device, then passed as CrossEntropyLoss's weight argument — this scales each class's contribution to the loss, forcing the network to pay more attention to getting rare classes (like 'building') right, instead of being dominated by common ones (like 'woodland').
- If class_weights is None, falls back to a plain unweighted CrossEntropyLoss(), which the code's own comment warns will likely push the model toward collapsing onto the single most common class.

metrics.py

Two evaluation functions used identically during training, validation, and testing.

`pixel_accuracy(predictions, targets)`

- The simplest possible metric: what fraction of all pixels did the model classify exactly right?
- Computed as (predictions == targets).sum() divided by targets.numel() (the total pixel count across the whole batch).
- Easy to interpret but can be misleading on imbalanced data — a model that always predicts 'woodland' could still score a high pixel accuracy just because woodland covers most of the image area.

`mean_iou(predictions, targets, num_classes)`

- IoU (Intersection over Union) for one class = (pixels predicted AND labeled as that class) divided by (pixels predicted OR labeled as that class) — a much stricter, shape-aware measure than raw accuracy.
- For each class id from 0 to num_classes-1, builds boolean masks of where predictions/targets equal that class, computes intersection and union via & and | on those boolean masks, and records intersection/union.
- If a class's union is 0 (it doesn't appear anywhere in this batch, in either predictions or ground truth), it's skipped entirely rather than counted as a 0 — this avoids unfairly punishing the score for classes that simply weren't present in a given batch.
- Returns the plain average of whatever per-class IoUs were collected; if none were (an empty batch edge case), returns 0.0.
- This is the metric train.py actually uses to decide which checkpoint is 'best', specifically because it's far more sensitive to poor performance on rare classes than pixel accuracy is.

train.py

The main entry point that actually trains SimpleUNet and saves the best checkpoint.

This is where every other support module gets wired together: dataset.py supplies data, model.py supplies the network, losses.py supplies the objective, metrics.py supplies the scorekeeping, and config.py supplies every setting used throughout.

Module-level setup

`torch.backends.cudnn.benchmark = True` lets cuDNN benchmark several convolution algorithms and pick the fastest one for this project's fixed 256x256 input size — a free speedup on CUDA, harmless (ignored) on CPU. `AMP_ENABLED = USE_AMP and DEVICE == "cuda"` makes sure mixed precision only actually turns on when both the config flag is set AND a CUDA GPU is present.

`run_one_epoch(model, loader, loss_fn, optimizer=None, scaler=None)`

- A single function handles both training and validation epochs — whether it trains depends entirely on whether optimizer is passed in (is_training = optimizer is not None).
- Sets model.train() (enables dropout/batchnorm-style training behavior, if any existed) or model.eval() accordingly.
- torch.set_grad_enabled(is_training) wraps the whole loop — gradients are only tracked during training, saving memory and computation during validation.
- For each batch: moves images/masks to DEVICE (non_blocking=True overlaps the CPU-to-GPU copy with other work instead of stalling), runs the forward pass inside torch.autocast (only actually active when AMP_ENABLED is True) to get logits and the loss.
- If training: zeroes gradients, then backpropagates and steps the optimizer — routed through the AMP GradScaler (scaler.scale/step/update) when mixed precision is on, since float16 gradients need loss-scaling to avoid underflowing to zero; falls back to a plain loss.backward()/optimizer.step() otherwise.
- Every batch (train or validation) accumulates loss, pixel_accuracy, and mean_iou; the function returns each metric averaged over all batches in the loader.

`main()`

- Creates CHECKPOINT_DIR if it doesn't exist, and prints which device/AMP setting this run is using.
- Builds train_dataset (augment=True) and valid_dataset (augment=False) — augmentation is intentionally only applied to training data, never validation, so validation scores reflect real, unaugmented performance.
- Wraps both in DataLoaders: train_loader shuffles each epoch, valid_loader doesn't (order doesn't matter for evaluation). persistent_workers keeps the worker processes alive between epochs instead of respawning them, saving overhead.
- Constructs the model, loss function (get_loss_function, passing CLASS_WEIGHTS), Adam optimizer, and the AMP GradScaler.
- Warns in the console if CLASS_WEIGHTS is None, nudging you to run compute_class_weights.py first.
- Main loop: for each of NUM_EPOCHS, calls run_one_epoch once with the training loader+optimizer, then again with the validation loader and optimizer=None (i.e. evaluation only). Prints both sets of loss/accuracy/IoU.
- Tracks best_val_iou across epochs; whenever the current epoch's validation mean IoU beats the previous best, saves model.state_dict() to checkpoints_landcover/best_model.pt — this is the exact file predict.py later loads. Using val_iou (not val_loss or train performance) to pick the checkpoint means the saved model is the one that generalizes best to unseen tiles, judged by the stricter per-class metric.

predict.py

Loads the trained checkpoint and runs inference — on the full test split, or on one arbitrary image.

`class_ids_to_rgb(class_id_mask: np.ndarray) -> np.ndarray`

- Converts a (H, W) array of raw class ids (0-4) into a (H, W, 3) uint8 color image, so a human can actually look at it.
- For each class index and its (name, color) pair in CLASS_COLORS, sets every pixel in the output where class_id_mask equals that index to that color.
- Used by both predict.py itself (to save colored PNGs) and imported into visualize.py / visualize_predictions.py for the same purpose — one canonical place for id-to-color conversion.

`load_model()`

- Constructs a fresh SimpleUNet(num_classes=NUM_CLASSES), moves it to DEVICE, then loads the saved weights from checkpoints_landcover/best_model.pt via model.load_state_dict(torch.load(...)).
- Calls model.eval() before returning — this disables any training-only behavior so inference is deterministic and doesn't accidentally update running statistics.

`predict_test_split(model)`

- The 'evaluate on everything' path — builds a SegmentationDataset(split="test", augment=False) and a DataLoader over it (shuffle=False, so tile order stays predictable).
- Rebuilds the same weighted loss function used in training, so the printed test loss is directly comparable to the train/valid loss numbers train.py already reported.
- For each batch, computes logits, loss, and predictions (argmax over the class dimension), and accumulates loss/pixel_accuracy/mean_iou exactly like run_one_epoch did.
- Also saves every individual predicted mask as a colored PNG (via class_ids_to_rgb) into OUTPUT_DIR, named <tile_name>_predicted_mask.png — tile_index is manually tracked in lockstep with the un-shuffled loader to look up the correct name for each prediction.
- After the loop, prints and writes the averaged loss/accuracy/mean_iou to test_metrics.csv — since LandCover.ai (unlike the companion DeepGlobe pipeline) ships real ground-truth masks for its test split too, this is described in the code's own comment as 'the truest measure' of the model, since test tiles were never seen during training or used to pick the checkpoint.

`predict_single_image(model, image_path)`

- The 'run on one arbitrary picture' path — for images that aren't part of the LandCover.ai dataset at all, so there's no ground truth to compare against and no metrics are computed.
- Opens the image, converts to RGB, resizes and normalizes it with the exact same preprocessing.py functions used everywhere else (so training and inference-time preprocessing stay consistent), then reshapes it into a (1, 3, H, W) batch tensor.
- Runs it through the model, argmaxes to get a predicted class-id mask, converts to a colored image, and saves it to OUTPUT_DIR as <original_filename>_predicted_mask.png.

`main()`

- Parses one optional --image argument. Loads the model once, then dispatches: if --image was given, calls predict_single_image with that path; otherwise runs the full predict_test_split path.
- This is why the module doc-comment shows both `python predict.py` and `python predict.py --image some_photo.jpg` as valid invocations.

visualize.py

A minimal pre-training sanity check: does the data actually look right?

`show_samples(split="train", num_samples=3)`

- Builds a SegmentationDataset for the given split with augment=False (so what you see is the raw, unmodified data), and creates a matplotlib grid with num_samples rows and 2 columns.
- For each of the first num_samples tiles, converts the image tensor back from (3, H, W) to (H, W, 3) for plotting, converts the mask tensor to a colored image with class_ids_to_rgb (imported from predict.py), and plots image next to mask.
- Saves the figure as sample_check_landcover.png in the current directory.
- Meant to be run once, early, to catch problems like mismatched image/mask pairs, wrong colors, or a broken preprocessing step, before spending time on a full training run.

Note this pulls class_ids_to_rgb from predict.py — a small but real coupling where a 'pre-training sanity check' script depends on the prediction script, since the color-conversion logic lives there rather than in a separate visualization utility module.

visualize_predictions.py

The post-training sanity check: how good are the model's predictions, visually?

Where visualize.py checks the raw data before training, this script checks the model's output after training (or after running predict.py) — for a handful of random test tiles, it shows three separate panels side by side (deliberately not blended/overlaid): the photo, the predicted mask, and the real ground-truth mask, so shapes and colors in columns 2 and 3 can be compared directly.

CLI arguments

`--num_samples` (default 6) controls how many random test tiles to show. `--seed` is optional — if set, random.seed(seed) makes the 'random' tile selection reproducible across runs, useful for comparing two different checkpoints on the exact same tiles.

main() walkthrough

- Parses arguments, seeds the RNG if requested, and ensures VISUALIZE_DIR exists.
- Loads the trained model via load_model() (from predict.py) and builds a test-split SegmentationDataset.
- Picks num_samples random, non-repeating indices with random.sample(range(len(test_dataset)), num_samples) — capped at the dataset size so it can't ask for more samples than exist.
- Builds a matplotlib grid with one row per sample and 3 columns; a small reshape guards the num_samples == 1 case so row/column indexing stays consistent either way.
- For each picked tile: runs the model forward (inside torch.no_grad(), since no training happens here), argmaxes to get the predicted mask, converts both the image tensor and both masks (predicted + ground truth) into displayable arrays, and plots all three into that row's three columns. The tile's own name is used as that row's y-axis label.
- Adds column titles ('actual (image)', 'predicted', 'label (ground truth)') to the first row only, and a shared legend across the top mapping each CLASS_COLORS color to its class name, so the colors in every panel are interpretable without cross-referencing config.py.
- Saves the whole figure to visualize_landcover/test_predictions_comparison.png.

In practice, this is the file you'd look at last — after check_mask_values, compute_class_weights, train.py, and predict.py have all run — to actually judge, by eye, whether the model is finding the right shapes in the right places.